

把前幾週組成一個遊戲

綠燈遮光加一次，紅燈遮光扣一次。30 秒內累積到 6，還要按結束才過關。

Start → 開始 30 秒

Finish → 判斷是否達 6

綠燈：新遮光 +1

紅燈：新遮光 -1

未遮 → 遮住 → 移開 → 再遮，才是下一次

OLED 顯示時間與 count；上限 6、下限 0

先用按鈕、光敏、RGB 與 OLED 玩完一局，再加入已確認的紙指針與失敗短聲。今天不新增零件，也不接網路。

做完要能說明三件事

- 為什麼遮住不放，不會一直加分？
- 螢幕、燈色、指針與紀錄，是否使用同一份結果？
- 玩家失敗和裝置故障，為什麼要分開？

跟著接線、做一局、看紀錄，再讀原理。後面的完整程式供開啟與查閱，不需要先背完所有函式。

還沒接線，也能先懂規則

一位同學說燈色，一位同學用手表示遮光，第三位記錄分數。不插電，先演下面四步。

起始 count = 0	動作	結果
綠燈	從未遮變成遮住	變 1
綠燈	持續遮住	還是 1
任一燈色	移開遮光物	還是 1，準備下一次
紅燈	再次遮住	回 0

最低 0、最高 6。到 6 不會自動贏：必須在 30 秒期限前按 **Finish 結束鍵**。當下不到 6，或到期還沒結束，都失敗。

先問自己：六次手勢是不是一定等於六次有效得分？不是，紅燈會扣分，遮住不放也不會重複算。

兩鍵同時按住或 Serial 送 x 是中止，不是玩家輸。裝置異常先停止處理，不要求玩家繼續玩。

先完成不含舵機的一局

操作順序	建議時間	完成時看什麼
規則、接線與設定	35 分鐘	每條線用途、設定不衝突
第一局、再看原理	35 分鐘	開始、一次遮光、紅扣分、結束
紀錄與中止測試	25 分鐘	result、原因、可追蹤的一局
加入指針 / 聲音	25 分鐘	原規則不變，各輸出一致
改造、討論與收尾	25 分鐘	只改一個時間參數及證據

另留約 20 分鐘休息。進度較慢時，把改造作延伸，不刪斷電與停止檢查。

先準備什麼？

Week 2 雙按鈕、Week 3 同配置光線基準、Week 5 RGB / OLED 的設定。完整版本另外需要 Week 4 的蜂鳴器及 Week 6 的舵機供電驗證。

前週講義完成，不代表實物已驗證。SG90 及帶載電源未通過時，舵機與蜂鳴器先不接，只完成低功率版本。低功率模組本身也必須符合已確認的接法。

同一條 GPIO 不兼兩份工作

功能	本週用什麼	程式欄位
讀光線	KY-018 的 S	PIN_LIGHT · 需 ADC 功能
開始 / 結束	兩顆獨立按鈕	PIN_START / PIN_FINISH
顯示紅綠燈	RGB 的三個色腳	PIN_RGB_R / G / B
顯示資料	OLED 的兩條通訊線	PIN_SDA / PIN_SCL
指針、聲音 (最後才接)	舵機、蜂鳴器	PIN_SERVO / PIN_BUZZER

低功率版使用 **8 個不同的訊號 GPIO**；完整版本再加 2 個。3V3、GND 不是這裡的訊號 GPIO 數量。

Week 2 的 GPIO4 曾接按鈕，後來已用來讀 KY；Week 6 的 STOP 按鈕也不能原封不動當本週兩鍵接線。**功能改了，程式與線也要一起核對。**

本週不接 DHT11，也不因手上有更多元件就全接上。請先將自己的核准 GPIO 填在上述表格旁，再追到板上絲印。

3V3 和 GND 分在不同孔組

USB 拔除，外部電池 OFF 並取出至少一顆。先收起舵機及蜂鳴器，板子放麵包板旁，避免擋住孔。

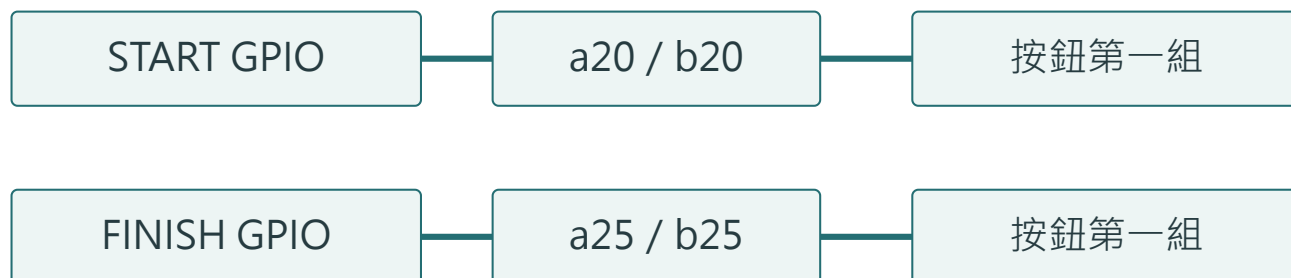


接線	目的
ESP32 3V3 → a6	提供低功率模組電源
ESP32 GND → a3	提供共同參考與返回路徑
OLED VDD → b6、GND → b3	沿用 Week 5 相容模組
KY 中間供電腳 → c6、－ → c3	沿用已核對的 KY 版本

同側 a~e 的同一列相通；第 3 與第 6 列不相通，a15 與 f15 也不是同一組。先用斷電通斷檔核對自己的板。

這是本頁的接線示例，不和既有作品不同版本的孔號混用。沒有蜂鳴只代表該次通斷觀察，不能證明極性或電流能力全部正確。

兩顆按鈕，兩條輸入線



各按鈕另一組 → 共地端子 → d3 / a3 → GND

一鍵一條 GPIO；不是把兩個輸入短在一起。

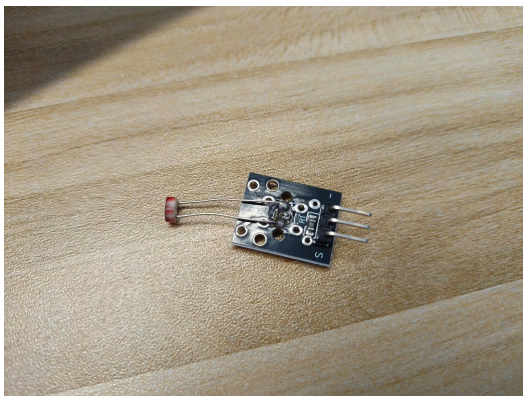
線	來源 → 目的
開始訊號	START GPIO → a20
開始按鈕	第一組接點 → b20；另一組 → 共地端子
結束訊號	FINISH GPIO → a25
結束按鈕	第一組接點 → b25；另一組 → 共地端子
共地	共地端子 → d3，與 a3 同組

b20 / b25 是導線延伸到按鈕的節點，不要求按鈕直接跨這些孔。每顆按鈕只取兩組不同接點；放開不通、按住才通。

程式用 `INPUT_PULLUP`，所以放開 HIGH、按住 LOW。按下的電流路徑是內部 3.3 V → 上拉電阻 → 輸入節點 → 按鈕 → GND，不是電源正極直接短接地。

兩鍵同時按住是軟體中止；不是具有電源隔離的急停。

光敏的位置也是設定的一部分



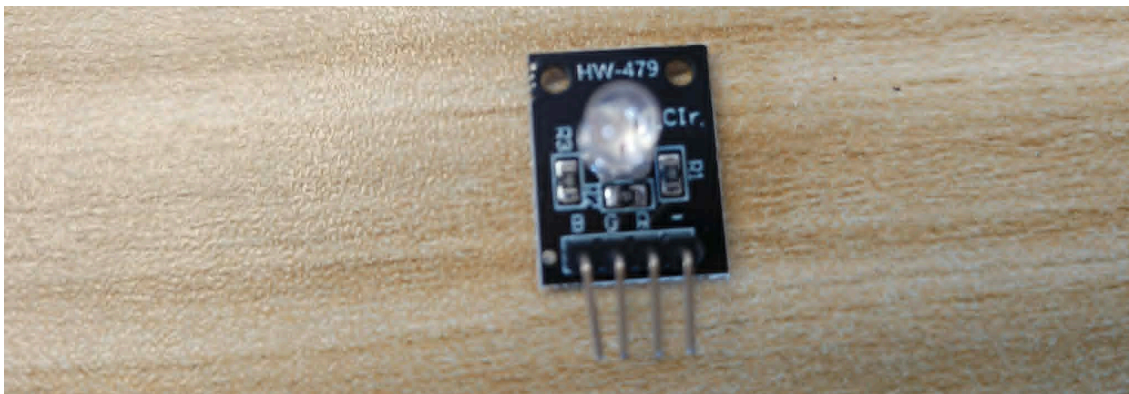
課程既有 KY 實物照片。使用已核對的 S、供電與 — 功能，不憑三針外觀猜腳位。

1. 保持斷電，KY 的 S → a15，再由 c15 → 設定表的 ADC GPIO。
2. 確認供電與地已按上一頁接好，e15 可保留作量測點。
3. 固定光敏的位置、方向與遮光物。不要把 RGB 或紙指針放在感光面旁。
4. 沿用**同一接法、同一場景**的「未遮 / 遮住」兩組 min、max。

若移了位置，或資料來自不同接法，就先回 Week 3 重新取得對應基準，不把不同日期的數字湊成一組。

T01 後來曾觀察到「遮光較小」，和早期方向不同。本程式支援兩種方向，但不能直接套早期門檻。raw 也不是 lux。

RGB 告訴玩家現在能不能遮



課程既有 HW-479 接點照片；依 Week 5 已確認的共同端、限流與有效準位使用。

RGB 功能	接法
R	核准的 PIN_RGB_R
G	核准的 PIN_RGB_G
B	核准的 PIN_RGB_B
共用腳	依 Week 5 確認型式；共陰的 — 才接共同地

本遊戲只亮紅或綠，B 仍由程式設定為關。三路各自需要限流；不可拿 WS2812 燈條當作同一模組。

先前個別三色有反應，可以引用該次紀錄；整合後仍需確認 OLED 說綠時，真正亮的是綠。若不一致，先中止，不把玩家依錯燈操作算成玩家失誤。

OLED 不只顯示分數

RUNNING

TIME 24 s 2/6

GREEN: COVER

STAGE1: NO SERVO/BEEP

畫面示意，不是實機截圖。

1. 斷電時把 SDA → 已確認的 PIN_SDA。
2. SCK / SCL → 已確認的 PIN_SCL，不與 SDA 對調。
3. 核對前面已接的 VDD / GND，不接外部舵機電源正極。
4. 程式沿用 Week 5 的位址與控制器設定；0x3C 不代表一定是哪一款晶片。

畫面依序讓你看：狀態、剩餘秒數與 count、目前規則、輸出版本。

看到 STAGE1: NO SERVO/BEEP，意思是本版本刻意不控制舵機與蜂鳴器。它沒有聲音並不是接壞了。

圖為版面示意。實際程式的兩個控制器選項是 1306 / 1315；只能套用先前已確認的 128×64 I2C 配置。

先選不含舵機的版本

開啟附錄的 `week07_traffic_light_challenge.ino`，先看檔案頂端。

設定	本次要填什麼
LESSON_STAGE	1：只用已確認的低功率元件
LOW_POWER_PROFILE_CONFIRMED	所用接法、模組、電流及GPIO均確認後才 true
光敏、兩鍵、RGB、OLED 腳位	本次設定表上的 8 個不同 GPIO
RGB_ON_LEVEL	Week 5 已確認的 HIGH 或 LOW
INDOOR_MIN / MAX	本次未遮光基準
SHADE_MIN / MAX	本次遮光基準
OLED 位址、OLED_CONTROLLER	沿用已確認畫面配置

舵機與蜂鳴器相關欄位保持 `false` / `-1`，實物也不接。公開原始碼預設 `LESSON_STAGE=0`，會阻擋操作；不是下載後就能直接動。

若只看到 `profile_missing`，先找缺少或衝突的欄位，不把所有 `false` 改成 `true`。軟體檢查不能替代電氣確認。

先看到 IDLE，再開始玩

1. 外部電池仍分離；拔 USB，暫時移除所有外接線，只留開發板。
2. 在 Arduino IDE 開啟本週完整程式。程式庫需 U8g2 2.36.15、ESP32Servo 3.2.1；版本 1 也需要兩者才能編譯。
3. 使用前週相同板型及已確認設定，Arduino-ESP32 基準 3.3.11；選實際 Port，編譯再上傳。
4. 拔 USB，按本週表恢復低功率接線。逐條核對後接 USB。
5. 開 Serial Monitor，選 **115200**。光敏保持未遮，兩鍵都放開。

預期 OLED 為 IDLE、TIME 30 s、0/6，RGB 關。Serial snapshot 應有 INDOOR、settled=true，表示目前分類穩定。

IDLE 就是待機；RUNNING 才是這一局正在進行。上傳完成只是程式寫入，不代表遊戲接線通過。

沒有畫面或仍 blocked，先查設定與錯誤紀錄，不開始接舵機。

按開始，再把手放開

先預測：按下 Start 後，是分數先加一，還是倒數先開始？

1. 保持未遮，Finish 已放開；按一下 Start，再放開。
2. 看 OLED：應從 IDLE 變 RUNNING，倒數開始，count 仍 0。
3. 看實物：第一段應綠燈。
4. 看 Serial：應出現 `event_type=start`、`reason=start_accepted`，並有新 `game_id`。

預期欄位（示例，不是實測）

```
event_type=start state=RUNNING value=0  
reason=start_accepted phase=GREEN
```

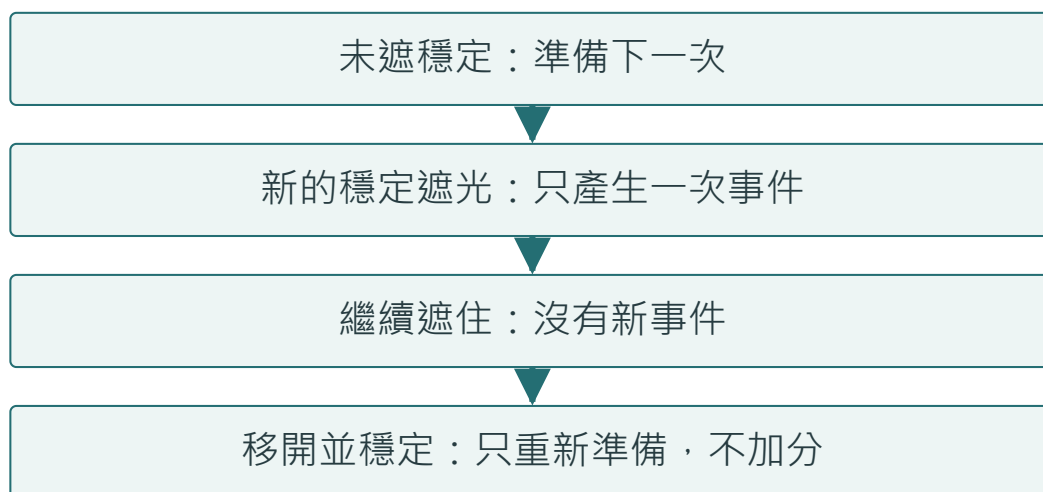
遊戲進行中再按 Start，不會延長時間或歸零。若開始被拒絕，先看 Finish 是否仍按著、是否未遮且穩定，不一直按到它開始。

低功率版本不需要 a。a 是完整版本準備舵機的命令，不是開始遊戲的指令。

綠燈遮住一秒，應加幾次？

在還有時間的綠燈段做一次；若本局結束，就依後頁的 z 流程重開，不趕著硬做。

1. 先未遮穩定，再遮住約一秒。
2. 看 count：應只加 1，不是每筆讀值都加。
3. 移開遮光物，等未遮穩定；這個動作不加分。
4. 下一個綠燈段，再遮住，才可能再加 1。



接著在紅燈做一次新的遮光，應扣 1，但最低 0。若一直遮住，從紅變綠也不自動補加分。

手勢太短可能沒被確認；接近換色時，以程式**確認新事件的時刻**判燈色，不是你認為手剛開始移動的時刻。

不到 6，也可以按 Finish

先故意在 count 不足 6 時按 Finish，觀察結算；這是規則測試，不是操作做錯。

操作 / 狀況	預期結果
期限前 Finish，當下不足 6	FAILED，時間與 count 凍結
期限前 Finish，當下是 6	SUCCESS，時間與 count 凍結
到 30 秒還沒有效 Finish	FAILED，即使 count 是 6

結果出現後 RGB 關，低功率版沒有失敗聲。繼續遮光不會修改已結算 count。

再玩一局

兩鍵放開、感測資料穩定可用，在 Serial 輸入 **z**。先回 IDLE、count=0，再保持未遮，重新按 Start。

z 不是重新開始：它只讓程式回待機。下次被接受的 Start 才增加 game_id。

先把第一局的 start、cover、result 三種紀錄找出來，再讀下一段原理。

螢幕變慢，時間也不應變慢

示例：開始 10000 ms，現在 12500 ms

經過 2500 ms → 剩餘 27500 ms

畫面向上取整 → 顯示 28 秒

計時看時間差；不是畫面更新一次才減一次。

```
經過時間 = 現在 millis() - 開始時 millis()  
剩餘時間 = 30000 - 經過時間 (最低 0)
```

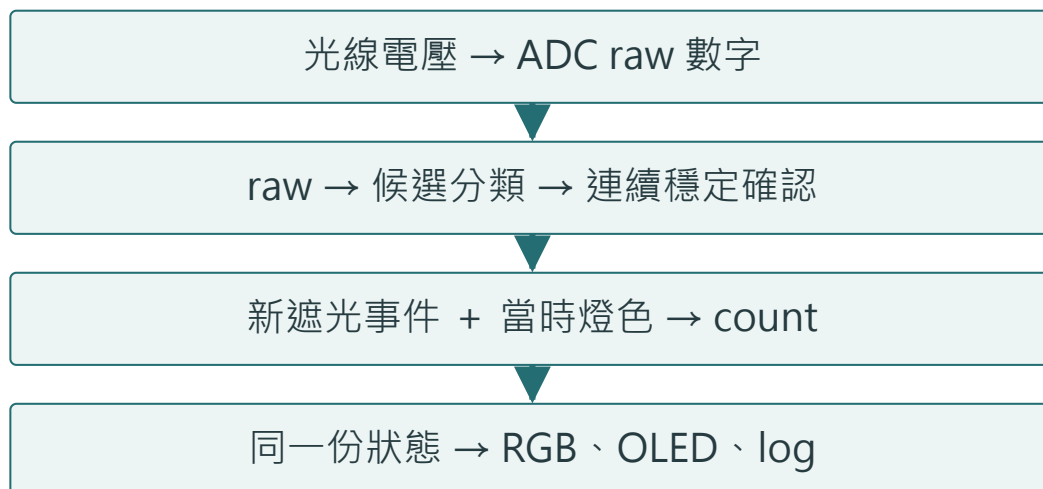
`millis()` 是這次開機後的毫秒數。1000 ms = 1 秒，不是日期或臺灣時間；重啟後不能和前一次直接相減。

紅綠燈也看經過時間：0 ~ 2999 ms 綠，3000 ~ 5999 ms 紅，6000 ~ 8999 ms 綠，以此交替。

OLED 約每 200 ms 更新，剩餘 1 ~ 1000 ms 都顯示 1 秒。畫面可能是上一幀，**不能用畫面仍有 1 秒推翻程式已到期的判定**。

程式不以 `delay` 等每一秒，但顯示與 `Serial` 函式仍可能耗時。這不是所有操作都零延遲。

讀值、分類、事件、count



名稱	你可以怎麼理解
raw	這次 ADC 讀到的數字，不是得分
INDOOR / SHADE	已確認的未遮 / 遮光分類
新遮光事件	從已穩定未遮，變成已穩定遮光
count	事件配上燈色後得到的有效次數

每 50 ms 嘗試取樣，1 秒約有 20 個取樣間隔；不表示人做了 20 次動作。

armed=true 是「已準備接受下一次遮光」。遮光計入後改 false，必須穩定未遮才重新準備。開機就一直遮住，不會先白得一次。

不要用 log 行數計分；snapshot 只是狀態快照，不是每一行都代表一次手勢。

中間的數字，先不要急著分類

假設同配置基準是未遮 300 ~ 320、遮光 900 ~ 920。這組是算式示例，不是全班設定。

$$\text{間隙} = 900 - 320 = 580$$

$$\text{下門檻} = 320 + 580 / 3 \text{ 的整數部分} = 513$$

$$\text{上門檻} = 320 + 2 \times 580 / 3 \text{ 的整數部分} = 706$$

$$\text{raw} \leq 513$$

候選未遮

$$513 < \text{raw} < 706$$

先不確認

$$\text{raw} \geq 706$$

候選遮光

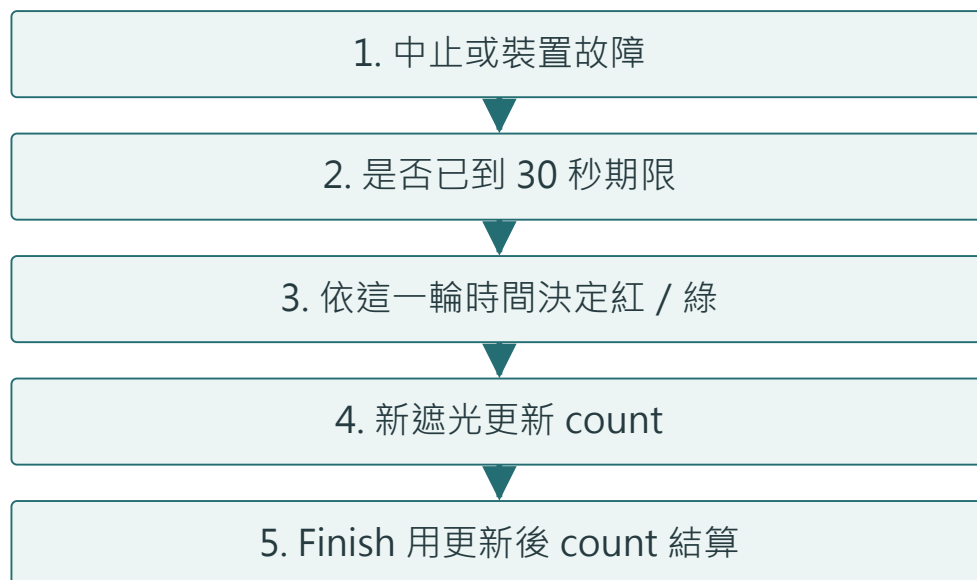
僅此示例：遮光數值較大。

同一候選分類要持續 **150 ms** 才確認。假設每 50 ms 正好取樣一次：0、50、100、150 ms 四筆，才跨滿 150 ms。

中間區會中斷尚未完成的確認，不把幾段零散的遮光時間累加。先前分類可能還顯示在畫面 / log，但 `settled=false` 時不能拿它當新事件。

若你的遮光數字較小，程式反轉兩端標籤。0 / 4095 在本遊戲視為不可用，不等於元件一定壞；合理 raw 也不保證線沒鬆。

先處理什麼，要寫成規則



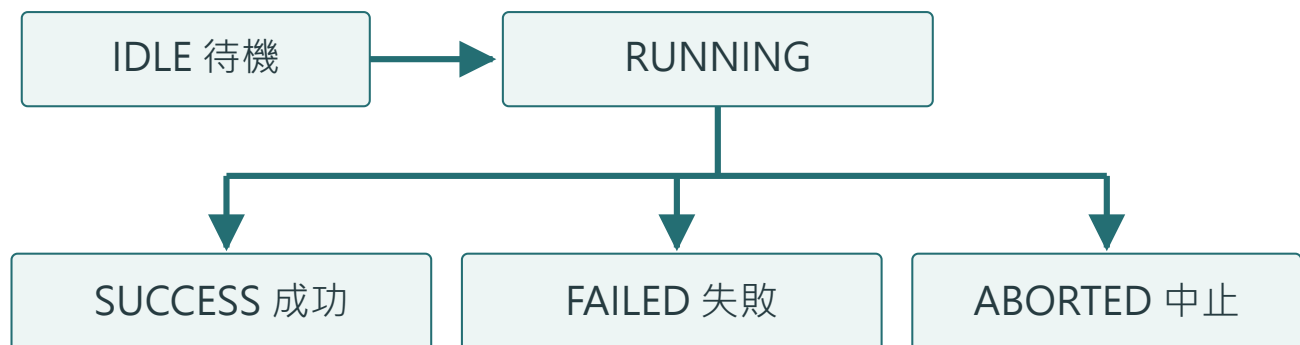
在同一輪處理中，**先計新遮光，再判 Finish**；但中止或到期優先。

假設沒有裝置故障	同輪事件	結果
經過 12000 ms，原 count=5	新遮光 + Finish	綠，加成 6，成功
經過 15000 ms，原 count=6	新遮光 + Finish	紅，扣成 5，失敗
經過 30000 ms，count=6	Finish	到期優先，失敗

「同輪」是程式一次處理的資料，不是保證兩個硬體訊號在同一微秒到達。這類邊界用程式測試，不要求學生用手精準做出。

完整版本中，同輪第六次加分後立即結算，可能已停止舵機脈波而指針尚未到 6；結果與實際位置仍要分開。

待機、進行、結束不能混在一起



主流程示意；待機也可能因故障中止。

畫面文字	此時能做什麼
IDLE 待機	條件符合才接受新的 Start
RUNNING 進行中	遮光計分、Finish 結算，Start 不重開
SUCCESS / FAILED	保存結果；符合條件後 z 回待機
ABORTED 中止	排除原因，c 確認，再 z 回待機

這叫**狀態機**：把此刻允許的操作寫清楚，避免每個按鈕各自修改同一局。

結果結算後再送 x，只停止輸出，不改寫已完成的 SUCCESS / FAILED。尚未結算時中止，才把這局記成 ABORTED。

按鍵放開不會自動復原。兩鍵一直按住時，也不能用 c、z 或 a 繞過中止條件。

找到同一局的三種紀錄

先找 `start`、`cover`、`result`，不要一開始就讀完每個欄位。

```
event_type=cover phase=RED  
count_before=5 count_after=4  
reason=red_penalty
```

以上是拆行示例；真正程式會在同一行印出更多欄位。

欄位	用來回答什麼
device_id、boot_id、game_id	哪個裝置、哪次開機、哪一局？
event_type、reason	發生什麼，為什麼？
count_before / after	這次事件到底有沒有改分？
uptime_ms、remaining_ms	開機後何時、這局還剩多少？
light_raw、settled、armed	當時感測分類可否支持這個事件？

`valid=true` 只是本程式的樣本新鮮與分類條件，不是感測精度保證。

`position_measured=false` 表示沒有讀回真實角度。

`boot_id` 是隨機識別值，不是絕對不重複的保證。本週 log 尚在 Serial，沒有自動存進資料庫；保存時標註裝置及測試日期。

用命令代替帶電拔線

先在低功率版新開一局，保持所有線固定。下列是**軟體故障注入**，不是所有硬體故障的模型。

1. Serial 送 f：之後以 -1 代替光線 raw。
2. 先看到 fault_injection；不是一送 f 就宣告玩家輸。
3. 持續約 2 秒無法穩定分類後，應 ABORTED，原因 light_unavailable；不發玩家失敗聲。
4. 送 r 停止注入，未遮並等新樣本穩定。r 不會自己復原遊戲。
5. 兩鍵放開，依序 c、z：回 IDLE，還沒開始新局。

另做一次「兩鍵同時按住」或 x 的人工中止；它不需要等 2 秒。

完整版本中止後先把舵機外部 OFF；要改線再拔 USB。不用短接電源、拔 GND 或卡住軸來製造故障。

樣本超過 200 ms 未更新也會中止。慢 I2C、資料無法分類、鬆線可能不同原因；先看 reason，不全部歸咎玩家。

先寫判斷，再翻頁

下列均假設沒有其他裝置故障；每個遮光事件已穩定確認，原版紅綠各 3 秒。

1. 五次綠燈新遮光，再一次紅燈新遮光，目前 count 是多少？此時 Finish 能成功嗎？
2. 紅燈新遮光後一直遮到綠燈，會自動加回一分嗎？
3. count 已是 6，但在經過 30000 ms 那輪才 Finish，結果是什麼？
4. raw 在 0 ms 是900、50 ms是900、100 ms是600、150 ms是900、200 ms是900。使用前面513 / 706門檻，到200 ms有新遮光事件嗎？
5. 看到 ABORTED，是否等於玩家只拿到0分？應先找哪個欄位？

回答時指出依據

不要只寫「有問題」或「失敗」。請寫出 count、時間條件、哪個樣本打斷確認，或哪個原因尚待排查。

事件、時間、故障分開算

題	參考解答
1	5-1=4；Finish 是 FAILED，不是六次手勢就贏。
2	不會。持續遮光沒有新的事件，必須先穩定未遮再遮。
3	FAILED ，因為到期優先，不再接受該輪 Finish。
4	沒有。100 ms 的600進中間區，中斷候選；150 ms重新開始，到200 ms只過50 ms。
5	不等於0分；是中止，不當作正常玩家結果。先找 result / snapshot 的 reason，再查感測新鮮度或通訊等證據。

第 4 題若後續250、300 ms仍是900，且之前已穩定未遮、armed=true，則300 ms可確認新遮光事件。

如果開機後一直遮住，沒有先穩定未遮，不能只因滿150 ms就憑空新增一次事件。

接下來才加入已分別驗證的舵機與蜂鳴器。若尚未完成驗證，保留本週低功率成果，不勉強接上。

同一個遊戲，不改勝負規則

同一個 count → OLED 與舵機角度命令

FAILED → 一次 200 ms 短聲命令

SUCCESS / ABORTED → 不發玩家失敗聲

所有結果 → 停指針脈波；外部仍須人工 OFF

先外部 OFF、取出至少一顆電池、拔 USB。僅在 Week 6 舵機供電、啟動與小行程，以及 Week 4 蜂鳴器介面已驗證時進行。

額外接線	來源 → 目的
舵機正極	降壓 VOUT+ → 安全端子 → 舵機 V+
舵機回流	舵機 GND → 負極端子 → VOUT-
共地及訊號	ESP32 GND → 負極端子；專用 GPIO → signal
蜂鳴器	專用 GPIO → Week 4 已核對的控制介面

舵機工作電流不走 GPIO、3V3 或麵包板低功率列。外部正極不接 ESP32 5Vin / USB VBUS。端子與線材須固定、絕緣並符合負載需求。

HW-508 的既有受限波形接法要保留串聯 1 kΩ，不能照商品「有源」字樣換成任意直驅。分項有反應不等於整合已通過。

a 是準備指針，不是開始計時

改程式時先斷兩種電源、移除外接線，再上傳：LESSON_STAGE=2，填舵機 / 供電 / 蜂鳴器的已確認設定。然後斷電恢復接線。

1. 接 USB、舵機外部仍 OFF，兩鍵放開且未遮穩定。
2. 只有已確認「工作電源 OFF 時接收 signal 不會異常回灌」的配置，才送 a 準備第0格。
3. 按既有已驗證順序外部 ON；觀察無異常，才按 Start。
4. a 後超過 **20秒** 未開始會中止。不是可以一直保持指針等人。
5. 新局跟前面一樣玩；綠加一、紅扣一，OLED與指針命令用同一 count。
6. 結果停止指針脈波；每局先外部 OFF，再做 z / 下一次 a。

若這個上電順序尚未確認，**停在低功率版**。程式的準備紀錄不是量到電池真的 OFF，不能當作電氣驗證。

FAILED 會要求一次200 ms短聲；SUCCESS與ABORTED不發失敗聲。實際聲長、停止延遲與PWM 資源相容性仍需整合實測。

燈光和指針不要替玩家遮光

手勢與 KY 區

RGB / OLED 區

光、手臂、紙片：不要互相干擾

指針掃動區：避開 KY

布局示意；重新擺位後需核對分類基準。

1. 固定 KY、桌燈和遮光物位置，不靠移動感測器玩遊戲。
2. 手完全離開感測區，觀察切紅 / 綠時 raw 是否明顯改變，是否錯判事件。
3. 確認指針的掃動、導線和同學手臂不遮住感光面。
4. 若整合後才誤判，先中止，改布局，再重新取得同配置基準。

比較前後時，只改元件相對位置，保留供電、程式與光源條件，並記錄變更。不用調門檻掩蓋未確認的接線問題。

紙指針最後停在哪，不一定等於已凍結的結果；停止脈波可能失去保持力，結算前也可能還沒到位。

看 OLED / log 的規則結果，另記實際位置。不要為了讓照片漂亮，在帶電時強扳指針。

同樣30秒，每色改成5秒

先用已完成的低功率版操作，舵機及蜂鳴器保持分離。這次只改紅綠切換時間，其餘不變。

在完整程式找到這一行：

```
const uint32_t GAME_MS=30000, COLOR_MS=3000,  
              SAMPLE_MS=50, STABLE_MS=150;
```

原始檔可能排在同一行；只把 `COLOR_MS=3000` 改成 `COLOR_MS=5000`。編譯、依原流程斷電移線後上傳，再斷電恢復接線。

原版：每 3 秒換色

綠	紅	綠	紅	綠	紅	綠	紅	綠	紅
---	---	---	---	---	---	---	---	---	---

修改：每 5 秒換色

綠	紅	綠	紅	綠	紅
---	---	---	---	---	---

0 秒

30 秒

兩者總綠燈都 15 秒，段數不同。

遊戲長度仍30秒、目標仍6、穩定時間仍150 ms、感測基準與布局不變。兩種設定都各有15秒綠燈、15秒紅燈，但分成的段數不同。

先預測：經過4秒時，原版和新版各是哪一色？不能只憑「切換較慢」就宣稱一定比較容易成功。

差的是切換時間，不是手勢資料

以下是在主機以假 I/O 餵給既有計分程式的**事件測試**。時間是遊戲開始後、已確認新遮光的時刻；不是未經濾波的原始手勢，也不是實機成績。

新事件時刻 ms	原版3秒切色 count	5秒切色 count
1000	1	1
2000	2	2
4000	1	3
7000	2	2
10000	1	3
13000	2	4

兩版均從0開始，14000 ms按 Finish，結果都是FAILED。4分比2分高，但仍未達6；不能只報「新版成功」。

程式、目標、事件時刻與Finish相同，只改COLOR_MS。此測試能核對計分邏輯，不能證明真實玩家在任一節奏下都比較好。

實物若由人重新玩兩局，手勢時間也會不同，請如實記下，不把差異全歸因於切換時間。

每1.5秒換色，其他不變

回到同一份低功率遊戲，把 COLOR_MS 改成1500。不要改 GAME_MS、TARGET、光線校正、去抖、穩定時間、接線或安全機制。

操作前先預測

1. 30秒會有幾個綠段、幾個紅段？總綠燈時間多少？
2. 經過1499 ms與1500 ms各是什麼顏色？
3. 使用上一頁完全相同的事件序列，依次count是多少？14000 ms Finish的結果呢？
4. 綠燈遮住後一直保持到紅燈，是否會自動扣分？

做完留下什麼

修改的那一行、預期表、程式測試或實物log，並標明資料來源。若做實物，使用同一布局，記下真正事件時刻，不硬湊成上一頁時間。

題目沒有要求增加元件或改變機械行程。下頁是參考解答，先完成預測再翻頁。

只改 COLOR_MS

```
const uint32_t GAME_MS=30000, COLOR_MS=1500,  
              SAMPLE_MS=50, STABLE_MS=150;
```

問題	參考結果
紅綠段數	$30000 \div 1500 = 20$ 段；綠10段、紅10段
總綠燈時間	$10 \times 1500 = 15000$ ms，仍是15秒
1499 / 1500 ms	前者綠，後者紅；1500正好開始新段
既定六事件的count	1、0、1、2、3、4
14000 ms Finish	count=4，FAILED
遮住跨色	不補算事件，不自動扣分；先穩定未遮再遮

這組主機事件測試最後和5秒版同樣4分，但過程不同：第二次事件曾扣回0。最後分數相同，不代表每一步相同。

此處沒有量到玩家反應、感測器延遲或機械位置。請把事件測試和實物結果分開保存。

先找是哪一段不一致

你看到什麼	先看哪裡	先不要做什麼
blocked	設定缺值、重複GPIO、兩組光線範圍是否重疊	全部改true
Start被拒絕	Finish放開、未遮穩定、完整版prepared	一直重啟
一直遮住只加一次	正常規則；移開等穩定再遮	把每筆raw都計分
OLED綠、實物紅	GPIO、準位、燈色對應	繼續評玩家對錯
ABORTED	reason、新樣本、分類、I2C	當作玩家0分
指針沒動	先外部OFF，再查命令與供電	強扳 / 升壓

蜂鳴器初始化、發聲或停止API故障可能阻擋啟動或鎖住後續操作，不能用 c / z 清掉硬體原因。先斷電檢查，必要時重新啟動。

程式不會抓到所有鬆線、假合理數值或畫面錯誤。每次只改一項，保留改前及改後紀錄，不為了通過就刪除中止條件。

留下一個能解釋的完整作品

證據	內容
接線與設定	這次板卡、模組、GPIO、基準與遊戲參數
一局紀錄	start、cover、result，保留裝置 / 開機 / 局次
正常與邊界	成功、提早失敗、到期；測試或實物分清
中止與復原	x / 兩鍵、f→r→c→z，指出原因
尚未完成	哪個元件或條件沒驗證，不冒充完成

不是每個項目都要另開一局；同一段完整紀錄可支持多項觀察，但不能把沒做過的項目打勾。

下週第一次專題報告，可以借用今天的分工：**輸入是什麼 → 程式怎麼判斷 → 輸出什麼 → 如何記錄與停止**。這裡不增加新評分或交件規定，正式要求依報告指引。

正常收尾先中止；完整版外部 OFF，再拔 USB，電表 OFF。斷電後才改線或收納。低功率版也要先拔 USB再拆線。

哪些是資料，哪些是本課規則？

- [Arduino-ESP32 ADC 官方文件](#)：raw、解析度及ADC API；raw不是lux。
- [Arduino Blink Without Delay](#)：以時間差管理工作，不用delay等整段時間。
- [U8g2 專案](#)：OLED函式與控制器配置。
- [ESP32Servo 專案](#)：控制命令不等於真實位置量測。

30秒、目標6、紅綠3秒、穩定150 ms、故障等待2秒，都是本課遊戲與程式選擇，不是感測器原廠規格。

照片沿用課程實物影像；圖是功能與資訊示意，不保證插針外觀順序。接線及供電限制依既有硬體交接記錄，不把未驗證舵機改寫為已通過。

這份重設稿的程式與事件檢查使用主機替代 I/O；沒有新上電、上傳韌體、聲長、負載或整合硬體測試。參考解答隨附在題目後，適合課堂共讀，不是保密考卷。

下頁起為同一支完整遊戲程式的附錄。不同分頁要合在一起使用；模式比較只改COLOR_MS，不必另外重寫遊戲。

week07_traffic_light_challenge

第 1 ~ 33 行。所有分頁合起來才是完整程式。

```
#include <Arduino.h>
#include <Wire.h>
#include <U8g2lib.h>
#include <ESP32Servo.h>
#include <esp_system.h>

// 0: blocked; 1: verified low-power game; 2: add verified servo + buzzer.
const int LESSON_STAGE = 0;
const char* DEVICE_ID = "BOARD-T01";
const bool LOW_POWER_PROFILE_CONFIRMED = false;
const bool SERVO_AND_POWER_PROFILE_CONFIRMED = false;
const bool BUZZER_PROFILE_CONFIRMED = false;
const int PIN_LIGHT = -1;
const int PIN_START = -1;
const int PIN_FINISH = -1;
const int PIN_RGB_R = -1;
const int PIN_RGB_G = -1;
const int PIN_RGB_B = -1;
const int RGB_ON_LEVEL = -1;
const int PIN_SDA = -1;
const int PIN_SCL = -1;
const int OLED_ADDRESS_7BIT = -1;
const int PIN_SERVO = -1;
const int PIN_BUZZER = -1;
const int BUZZER_ON_LEVEL = -1;
const bool BUZZER_USE_TONE = false;
const int BUZZER_SERIES_OHMS = -1; // HW-508 tone path requires a verified 1 kOhm resistor.
const uint32_t BUZZER_HZ = 2000;
const int SERVO_HZ = -1;
const int SERVO_MIN_US = -1;
const int SERVO_MAX_US = -1;
const int SAFE_MIN_ANGLE = -1;
const int SAFE_MAX_ANGLE = -1;
```

下一頁接續同一支程式，不另開草稿。

week07_traffic_light_challenge

第 34 ~ 61 行。所有分頁合起來才是完整程式。

```
const int ZERO_ANGLE = -1;
const int SIX_ANGLE = -1;
const int INDOOR_MIN = -1;
const int INDOOR_MAX = -1;
const int SHADE_MIN = -1;
const int SHADE_MAX = -1;

const uint32_t GAME_MS=30000, COLOR_MS=3000, SAMPLE_MS=50, STABLE_MS=150;
const uint32_t BUTTON_MS=30, UNSETTLED_LIMIT_MS=2000, SAMPLE_STALE_MS=200;
const uint32_t DISPLAY_MS=200, LOG_MS=500, BEEP_MS=200, PREPARE_LIMIT_MS=20000;
const int TARGET=6;
// ACK is not a controller ID. Select after electrical and fixed-screen checks.
#ifndef OLED_CONTROLLER
#define OLED_CONTROLLER 1306
#endif
#if OLED_CONTROLLER == 1306
U8G2_SSD1306_128X64_NONAME_F_HW_I2C oled(U8G2_R0,U8X8_PIN_NONE,PIN_SCL,PIN_SDA);
#elif OLED_CONTROLLER == 1315
U8G2_SSD1315_128X64_NONAME_F_HW_I2C oled(U8G2_R0,U8X8_PIN_NONE,PIN_SCL,PIN_SDA);
#else
#error Unsupported OLED_CONTROLLER: use a verified 1306 or 1315 configuration.
#endif
Servo pointer;
enum GameState { IDLE, RUNNING, SUCCESS, FAILED, ABORTED };
enum LightState { UNKNOWN=-1, INDOOR=0, SHADE=1 };
GameState state=IDLE;
LightState stable=UNKNOWN,candidate=UNKNOWN;
bool ready=false,shadeHigh=true,lightSettled=false,eventArmed=false,hadSample=false;
```

下一頁接續同一支程式，不另開草稿。

week07_traffic_light_challenge

第 62 ~ 91 行。所有分頁合起來才是完整程式。

```
bool unsettled=true,injectedFault=false,ioFault=false,cleared=false,prepared=false;
bool greenPhase=false,buzzerOn=false,dirty=true;
bool buzzerFault=false;
int lowerThreshold=-1,upperThreshold=-1,lightRaw=-1,count=0,angleCommand=-1;
uint32_t bootId=0,gameId=0,startedAt=0,frozenRemaining=GAME_MS,candidateAt=0;
uint32_t unsettledAt=0,lastSample=0,lastDraw=0,lastLog=0,beepAt=0,preparedAt=0;
const char* reason="boot_idle";
struct Button {
    bool raw=false,candidate=false,stable=false,edge=false;
    uint32_t changedAt=0;
    void update(bool pressed,uint32_t now){
        raw=pressed;edge=false;
        if(pressed!=candidate){candidate=pressed;changedAt=now;}
        if(candidate!=stable && now-changedAt>=BUTTON_MS){stable=candidate;edge=stable;}
    }
    bool released(uint32_t now)const{return !raw&&!stable&&!candidate&&now-
changedAt>=BUTTON_MS;}
};
Button startButton,finishButton;

bool profileReady(){
    if(LESSON_STAGE<1||LESSON_STAGE>2||!LOW_POWER_PROFILE_CONFIRMED)return false;
    if(RGB_ON_LEVEL!=LOW&&RGB_ON_LEVEL!=HIGH)return false;
    if(OLED_ADDRESS_7BIT<8||OLED_ADDRESS_7BIT>119)return false;
    if(INDOOR_MIN<=0||SHADE_MIN<=0||INDOOR_MAX>=4095||SHADE_MAX>=4095||
        INDOOR_MIN>INDOOR_MAX||SHADE_MIN>SHADE_MAX)return false;
    shadeHigh=INDOOR_MAX<SHADE_MIN;
    if(!shadeHigh && !(SHADE_MAX<INDOOR_MIN))return false;
    const int lowEnd=shadeHigh?INDOOR_MAX:SHADE_MAX;
    const int highBegin=shadeHigh?SHADE_MIN:INDOOR_MIN;
    if(highBegin-lowEnd<3)return false;
```

下一頁接續同一支程式，不另開草稿。

week07_traffic_light_challenge

第 92 ~ 114 行。所有分頁合起來才是完整程式。

```

lowerThreshold=lowEnd+(highBegin-lowEnd)/3;
upperThreshold=lowEnd+2*(highBegin-lowEnd)/3;
const int pins[]={PIN_LIGHT,PIN_START,PIN_FINISH,PIN_RGB_R,PIN_RGB_G,PIN_RGB_B,
    PIN_SDA,PIN_SCL,PIN_SERVO,PIN_BUZZER};
const int size=LESSON_STAGE==2?10:8;
for(int i=0;i<size;i++){if(pins[i]<0)return false;for(int
j=0;j<i;j++)if(pins[i]==pins[j])return false;}
if(LESSON_STAGE==2 && (!SERVO_AND_POWER_PROFILE_CONFIRMED||!BUZZER_PROFILE_CONFIRMED||
    (BUZZER_USE_TONE ? BUZZER_SERIES_OHMS!=1000 :
    (BUZZER_ON_LEVEL!=LOW&&BUZZER_ON_LEVEL!=HIGH))||SERVO_HZ<10|SERVO_HZ>400||
    SERVO_MIN_US<500|SERVO_MAX_US>2500|SERVO_MIN_US>=SERVO_MAX_US||
    SAFE_MIN_ANGLE<0|SAFE_MAX_ANGLE>180|SAFE_MIN_ANGLE>ZERO_ANGLE||
    ZERO_ANGLE>=SIX_ANGLE|SIX_ANGLE>SAFE_MAX_ANGLE))return false;
return true;
}
const char* stateName(){switch(state){case IDLE:return "IDLE";case RUNNING:return
"RUNNING";
    case SUCCESS:return "SUCCESS";case FAILED:return "FAILED";default:return "ABORTED";}}
const char* lightName(){return
stable==SHADE?"SHADE":stable==INDOOR?"INDOOR":"UNKNOWN";}
bool terminal(){return state==SUCCESS||state==FAILED||state==ABORTED;}
uint32_t remainingAt(uint32_t now){return state==RUNNING?(now-startedAt>=GAME_MS?
0:GAME_MS-(now-startedAt)):frozenRemaining;}
uint32_t displaySeconds(uint32_t ms){return (ms+999)/1000;}
bool bothReleased(uint32_t now){return
startButton.released(now)&&finishButton.released(now);}
bool fresh(uint32_t now){return hadSample&&now-lastSample<=SAMPLE_STALE_MS;}
bool oledAck(){Wire.beginTransmission(OLED_ADDRESS_7BIT);return
Wire.endTransmission()==0;}

```

下一頁接續同一支程式，不另開草稿。

week07_traffic_light_challenge

第 115 ~ 141 行。所有分頁合起來才是完整程式。

```
void rgbOff(){const int off=1-
RGB_ON_LEVEL;digitalWrite(PIN_RGB_R,off);digitalWrite(PIN_RGB_G,off);digitalWrite(PIN_R
GB_B,off);}
void rgbPhase(){rgbOff();digitalWrite(greenPhase?PIN_RGB_G:PIN_RGB_R,RGB_ON_LEVEL);}
void silence(){
  if(LESSON_STAGE==2){
    if(BUZZER_USE_TONE){
      if(!ledcWrite(PIN_BUZZER,0)){
        ledcDetach(PIN_BUZZER);digitalWrite(PIN_BUZZER,LOW);pinMode(PIN_BUZZER,OUTPUT);
        if(!buzzerFault)Serial.println("event_type=buzzer_fault reason=tone_stop_failed
restart_required=true");
        buzzerFault=true;
      }
    }else digitalWrite(PIN_BUZZER,1-BUZZER_ON_LEVEL);
  }
  buzzerOn=false;
}
bool initializeBuzzer(){
  digitalWrite(PIN_BUZZER,BUZZER_USE_TONE?LOW:1-
BUZZER_ON_LEVEL);pinMode(PIN_BUZZER,OUTPUT);
  if(BUZZER_USE_TONE&&!ledcAttach(PIN_BUZZER,BUZZER_HZ,10))return false;
  silence();return !buzzerFault;
}
bool startBeep(uint32_t now){
  if(buzzerFault)return false;
  if(BUZZER_USE_TONE){
    if(ledcWriteTone(PIN_BUZZER,BUZZER_HZ)==0){buzzerFault=true;silence();return false;}
  }else digitalWrite(PIN_BUZZER,BUZZER_ON_LEVEL);
  buzzerOn=true;beepAt=now;return true;
}
void detachPointer(){if(LESSON_STAGE==2)
{if(pointer.attached())pointer.detach();digitalWrite(PIN_SERVO,LOW);}prepared=false;}
```

下一頁接續同一支程式，不另開草稿。

week07_traffic_light_challenge

第 142 ~ 165 行。所有分頁合起來才是完整程式。

```
int angleForCount(int n){return ZERO_ANGLE+(SIX_ANGLE-ZERO_ANGLE)*n/TARGET;}
void logEvent(const char* type,const char* why,uint32_t now,int before){
    Serial.printf("device_id=%s boot_id=%08lx game_id=%lu event_type=%s state=%s value=%d
unit=count "
    "valid=%s reason=%s uptime_ms=%lu phase=%s light_raw=%d light_state=%s settled=%s
armed=%s "
    "count_before=%d count_after=%d remaining_ms=%lu result=%s angle_command=%d
position_measured=false\n",
    DEVICE_ID,(unsigned long)bootId,(unsigned long)gameId,type,stateName(),count,
    fresh(now)&&lightSettled&&!injectedFault?"true":"false",why,(unsigned long)now,
    state==RUNNING?
(greenPhase?"GREEN":"RED"):"OFF",lightRaw,lightName(),lightSettled?"true":"false",
    eventArmed?"true":"false",before,count,(unsigned
long)remainingAt(now),stateName(),angleCommand);
}
void writePointer(uint32_t now){
    if(LESSON_STAGE==2&&pointer.attached()){

angleCommand=angleForCount(count);pointer.write(angleCommand);logEvent("pointer_command
","count_mapping",now,count);
    }
}
void settle(GameState target,const char* why,uint32_t now){
    if(terminal()){if(target==ABORTED){silence();detachPointer();rgbOff();}return;}
    if(target!=ABORTED&&state!=RUNNING)return;
    frozenRemaining=remainingAt(now);state=target;reason=why;cleared=false;dirty=true;
    rgbOff();silence();detachPointer();
    if(target==FAILED&&LESSON_STAGE==2&&!startBeep(now)){
        state=ABORTED;reason="buzzer_failed_restart_required";
    }
    logEvent("result",reason,now,count);
}
```

下一頁接續同一支程式，不另開草稿。

week07_traffic_light_challenge

第 166 ~ 192 行。所有分頁合起來才是完整程式。

```

}
// Confirm an event only after a classified value remains for STABLE_MS.
// Middle-band readings keep the displayed previous state, but cannot confirm or rearm.
bool sampleLight(int raw,uint32_t now){
    lightRaw=raw;lastSample=now;hadSample=true;
    LightState next=UNKNOWN;
    if(raw>0&&raw<4095){if(raw<=lowerThreshold)next=shadeHigh?INDOOR:SHADE;
        else if(raw>=upperThreshold)next=shadeHigh?SHADE:INDOOR;}
    if(next==UNKNOWN){candidate=UNKNOWN;candidateAt=now;lightSettled=false;
        if(raw<=0||raw>=4095){stable=UNKNOWN;eventArmed=false;}
    }else{
        if(next!=candidate){candidate=next;candidateAt=now;}
        lightSettled=now-candidateAt>=STABLE_MS;
        if(lightSettled){
            const bool changed=stable!=next;stable=next;
            if(next==INDOOR)eventArmed=true;
            if(next==SHADE&&changed){const bool
event=eventArmed;eventArmed=false;unsettled=false;return event;}
        }
    }
    if(lightSettled)unsettled=false;
    else if(!unsettled){unsettled=true;unsettledAt=now;}
    return false;
}

bool persistentFault(uint32_t now){return unsettled&&now-
unsettledAt>=UNSETTLED_LIMIT_MS;}

void stepGame(uint32_t now,bool startEdge,bool finishEdge,bool coverEdge,bool
abortRequest){
    // One snapshot, fixed priority: abort -> deadline -> color -> score -> finish.
    if(abortRequest||ioFault||buzzerFault||(state==RUNNING&&
(!fresh(now)||persistentFault(now)))){

```

下一頁接續同一支程式，不另開草稿。

week07_traffic_light_challenge

第 193 ~ 216 行。所有分頁合起來才是完整程式。

```
settle(ABORTED, abortRequest?"manual_or_both_buttons":buzzerFault?"buzzer_failed_restart_required":
    ioFault?"display_ack_failed":"light_unavailable", now); return;
}
if(state==IDLE){
    if(LESSON_STAGE==2&&prepared&&now-preparedAt>=PREPARE_LIMIT_MS)
{settle(ABORTED, "prepare_timeout", now); return;}
    if(startEdge){
        if(!finishButton.raw&&!finishButton.stable&&finishButton.released(now)&&fresh(now)&&
            lightSettled&&stable==INDOOR&&!injectedFault&&(LESSON_STAGE==1||prepared)){

state=RUNNING; gameId++; startedAt=now; frozenRemaining=GAME_MS; count=0; greenPhase=true;

reason="start_accepted"; eventArmed=true; rgbPhase(); dirty=true; logEvent("start", reason, now, 0);
        }else logEvent("start_rejected", "release_finish_uncover_or_prepare", now, count);
        }
        return;
    }
    if(state!=RUNNING) return;
    if(now-startedAt>=GAME_MS){settle(FAILED, "deadline", now); return;}
    const bool nextGreen=((now-startedAt)/COLOR_MS)%2==0;
    if(nextGreen!=greenPhase)
{greenPhase=nextGreen; rgbPhase(); dirty=true; logEvent("phase", "clock_phase", now, count);}
    if(coverEdge){const int before=count;
        count=greenPhase?(count<TARGET?count+1:TARGET):(count>0?count-1:0);
        dirty=true; logEvent("cover", greenPhase?"green_credit":"red_penalty", now, before);
        if(count!=before) writePointer(now);
    }
    if(finishEdge) settle(count==TARGET?
SUCCESS:FAILED, count==TARGET?"finish_target":"finish_below_target", now);
```

下一頁接續同一支程式，不另開草稿。

week07_traffic_light_challenge

第 217 ~ 239 行。所有分頁合起來才是完整程式。

```

}
void handleCommand(char c,uint32_t now){
    if(c=='\r' || c=='\n' || c==' ')return;
    if(c=='x'){settle(ABORTED,"manual_abort",now);return;}
    if(c=='f')
{injectedFault=true;sampleLight(-1,now);logEvent("fault_injection","software_missing_no
t_wire_removal",now,count);return;}
    if(c=='r')
{injectedFault=false;logEvent("injection_removed","wait_new_stable_sample",now,count);r
eturn;}

    if(c=='c'&&state==ABORTED&&bothReleased(now)&&fresh(now)&&lightSettled&&!injectedFault&
&!buzzerFault){

        ioFault=!oledAck();cleared=!ioFault;logEvent("clear",cleared?"reset_required":"display_
still_unresponsive",now,count);return;
    }

    if(c=='z'&&terminal())&&bothReleased(now)&&fresh(now)&&lightSettled&&!injectedFault&&!io
Fault&&!buzzerFault&&
        (state!=ABORTED || cleared)){
        silence();detachPointer();state=IDLE;count=0;angleCommand=-1;frozenRemaining=GAME_MS;

        cleared=false;reason="reset_idle_not_started";dirty=true;logEvent("reset",reason,now,0)
;return;
    }
    // Operator must have external servo power OFF and the approved power sequence.

    if(c=='a'&&LESSON_STAGE==2&&state==IDLE&&!prepared&&bothReleased(now)&&fresh(now)&&ligh
tSettled&&
        stable==INDOOR&&!ioFault&&!injectedFault&&!buzzerFault){
        pointer.setPeriodHertz(SERVO_HZ);pointer.attach(PIN_SERVO,SERVO_MIN_US,SERVO_MAX_US);
        if(!pointer.attached()){settle(ABORTED,"attach_failed",now);return;}
        prepared=true;preparedAt=now;count=0;writePointer(now);dirty=true;
        logEvent("prepare","operator_power_off_acknowledged_not_sensed",now,count);return;
    }
    logEvent("command_rejected","state_or_guard",now,count);

```

下一頁接續同一支程式，不另開草稿。

week07_traffic_light_challenge

第 240 ~ 265 行。所有分頁合起來才是完整程式。

```

}
void drawSnapshot(uint32_t now){
  if(ioFault)return;
  if(!oledAck()){ioFault=true;settle(ABORTED,"display_ack_failed",now);return;}
  char s[26];oled.clearBuffer();oled.setFont(u8g2_font_6x12_tf);
  oled.drawStr(0,12,stateName());
  snprintf(s,sizeof(s),"TIME %lu s  %d/6",(unsigned
long)displaySeconds(remainingAt(now)),count);oled.drawStr(0,28,s);
  const char* hint=state==RUNNING?(greenPhase?"GREEN: COVER":"RED: NO COVER"):
    state==SUCCESS?"PASS / RESET z":state==FAILED?"FAIL / RESET z":state==ABORTED?"ABORT
/ CHECK LOG":
    LESSON_STAGE==1?"UN COVER / START":prepared?"READY / START":"POWER OFF / ARM a";
  oled.drawStr(0,44,hint);oled.drawStr(0,60,LESSON_STAGE==1?"STAGE1: NO
SERVO/BEEP":pointer.attached()?"POINTER: COMMAND ONLY":"POINTER: NO SIGNAL");
  oled.sendBuffer();
  if(!oledAck()){ioFault=true;settle(ABORTED,"display_ack_failed",millis());}
}
void setup(){
  Serial.begin(115200);delay(1000);
  if(!profileReady()){Serial.println("week=7 status=blocked
reason=profile_missing");return;}
  ready=true;bootId=esp_random();unsettledAt=millis();
  for(int p:{PIN_RGB_R,PIN_RGB_G,PIN_RGB_B}){digitalWrite(p,1-
RGB_ON_LEVEL);pinMode(p,OUTPUT);}
  pinMode(PIN_START,INPUT_PULLUP);pinMode(PIN_FINISH,INPUT_PULLUP);
  startButton.changedAt=finishButton.changedAt=millis();
  analogReadResolution(12);analogSetPinAttenuation(PIN_LIGHT,ADC_11db);
  if(LESSON_STAGE==2){digitalWrite(PIN_SERVO,LOW);pinMode(PIN_SERVO,OUTPUT);
    if(!initializeBuzzer()){
      ready=false;Serial.println("week=7 status=blocked
reason=buzzer_init_failed");return;
    }}
}

```

下一頁接續同一支程式，不另開草稿。

week07_traffic_light_challenge

第 266 ~ 286 行。所有分頁合起來才是完整程式。

```
if(!Wire.begin(PIN_SDA,PIN_SCL,100000))
{ioFault=true;settle(ABORTED,"i2c_begin_failed",millis());}
else {Wire.setTimeout(20);if(!oledAck())
{ioFault=true;settle(ABORTED,"display_ack_failed",millis());}

else{oled.setI2CAddress(OLED_ADDRESS_7BIT*2);oled.setBusClock(100000);oled.begin();Wire
.setTimeout(20);}}
Serial.printf("week=7 stage=%d duration_ms=%lu stable_ms=%lu lower=%d upper=%d
shade_high=%s "
"commands=a,x,f,r,c,z no_physical_feedback=true\n",LESSON_STAGE,(unsigned
long)GAME_MS,
(unsigned long)STABLE_MS,lowerThreshold,upperThreshold,shadeHigh?"true":"false");
}
void loop(){
if(!ready)return;
uint32_t now=millis();startButton.update(digitalRead(PIN_START)==LOW,now);
finishButton.update(digitalRead(PIN_FINISH)==LOW,now);
bool cover=false;if(!hadSample||now-
lastSample>=SAMPLE_MS)cover=sampleLight(injectedFault?-1:analogRead(PIN_LIGHT),now);
// Drain at most one command; input work remains bounded per iteration.
if(Serial.available())handleCommand((char)Serial.read(),now);
// A command can invalidate the sample taken earlier in this same iteration.
if(injectedFault||!lightSettled)cover=false;

stepGame(now,startButton.edge,finishButton.edge,cover,startButton.raw&&finishButton.raw
);
if(buzzerOn&&now-beepAt>=BEEP_MS)silence();
// Skip display I/O during the short beep; checking its expiry takes priority.
if(!buzzerOn&&(dirty||now-lastDraw>=DISPLAY_MS))
{lastDraw=now;dirty=false;drawSnapshot(now);}
now=millis();
```

下一頁接續同一支程式，不另開草稿。

week07_traffic_light_challenge

第 287 ~ 294 行。所有分頁合起來才是完整程式。

```
// I2C is synchronous. Recheck safety/deadline with NEW time, without replaying edges.
const bool abortNow=digitalRead(PIN_START)==LOW&&digitalRead(PIN_FINISH)==LOW;

if(abortNow||ioFault)settle(ABORTED,abortNow?"both_buttons_after_display":"display_ack_
failed",now);
else if(state==RUNNING&&!fresh(now))settle(ABORTED,"sample_stale_after_display",now);
else if(state==RUNNING&&now-
startedAt>=GAME_MS)settle(FAILED,"deadline_after_display",now);
if(buzzerOn&&now-beepAt>=BEEP_MS)silence();
if(now-lastLog>=LOG_MS){lastLog=now;logEvent("snapshot",reason,now,count);}
}
```

這支程式到此結束。原始檔保留未確認腳位的保護值；不要把編譯成功當成實機通過。